

CLINIC QUEUE MANAGEMENT SYSTEM

1. Project overview

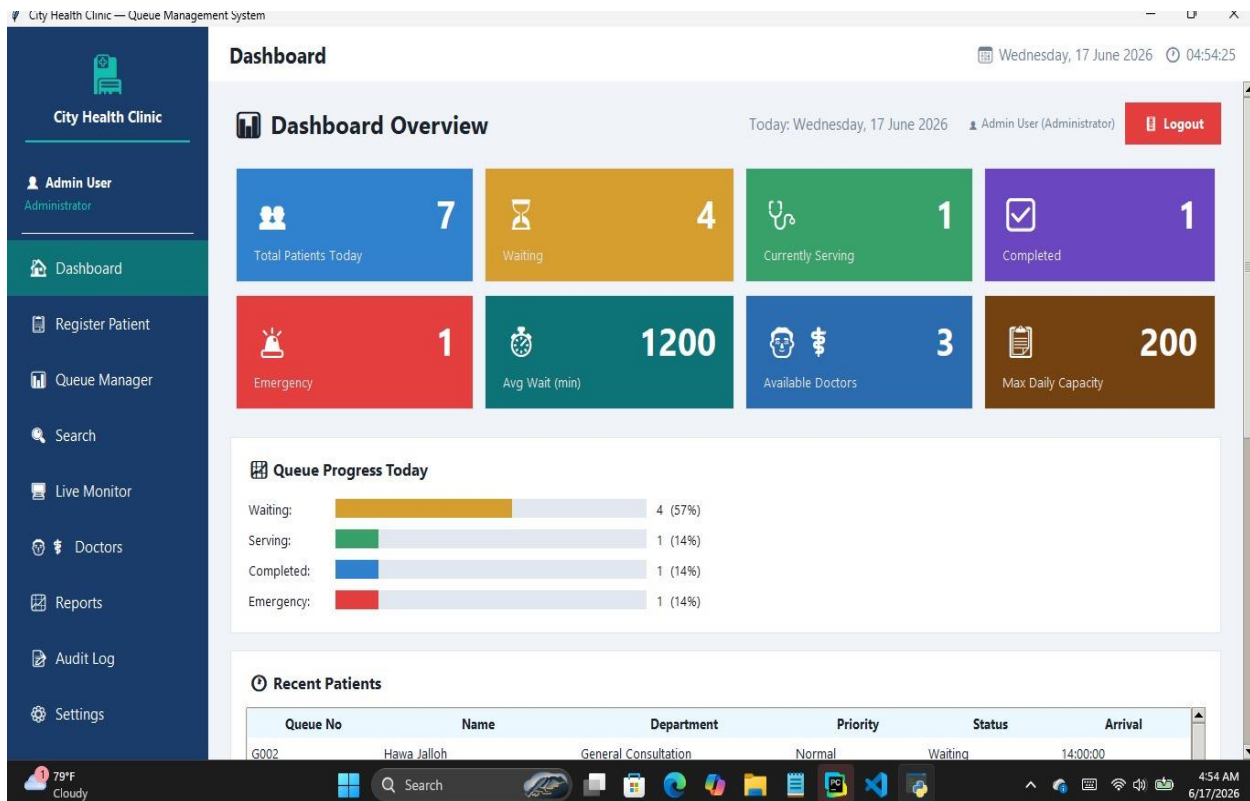
1.1 Problem Description

Public health facilities across Sierra Leone, particularly community clinics and primary healthcare centres, continue to rely on manual, paper-based methods for managing patient queues. Patients typically register their names on handwritten sheets or wait in unstructured physical lines, with no clear method of tracking arrival order, prioritising urgent cases, or recording how many people have been served in a day.

This manual approach creates several recurring problems. First, there is no reliable mechanism to prioritise emergency cases, elderly patients, or patients with urgent conditions over routine walk-ins, since the queue is purely based on physical arrival order. Second, clinic staff have no easy way to track which doctor is available, which department a patient should be directed to, or how long patients have been waiting. Third, there is little to no historical record-keeping for daily patient volume, peak hours, or department workload, which makes it difficult for clinic administrators to plan staffing or reorder resources. Finally, because everything is recorded on paper, there is a heightened risk of lost records, illegible handwriting, and an inability to quickly search for a specific patient's status.

These inefficiencies directly affect patient experience and clinical outcomes. Patients in genuine medical emergencies can be made to wait behind routine cases simply because they joined the line later, and clinic staff spend valuable time on administrative confusion rather than patient care.

1.2 System Explanation



The Clinic Queue Management System is a desktop Graphical User Interface (GUI) application built in Python using the tkinter library. It digitises the entire patient queue workflow for a small to mid-sized health facility, replacing the paper queue book with a structured, role-based, and auditable digital system.

The system supports three categories of staff, each with a distinct login and a different set of permissions appropriate to their role:

- Administrator – full access to all modules, including user management, system settings, and the audit log.
- Receptionist – can register new patients, manage the queue, search records, and generate reports.
- Doctor – can view the live queue monitor, search patient records, and see doctor availability, without access to administrative functions.

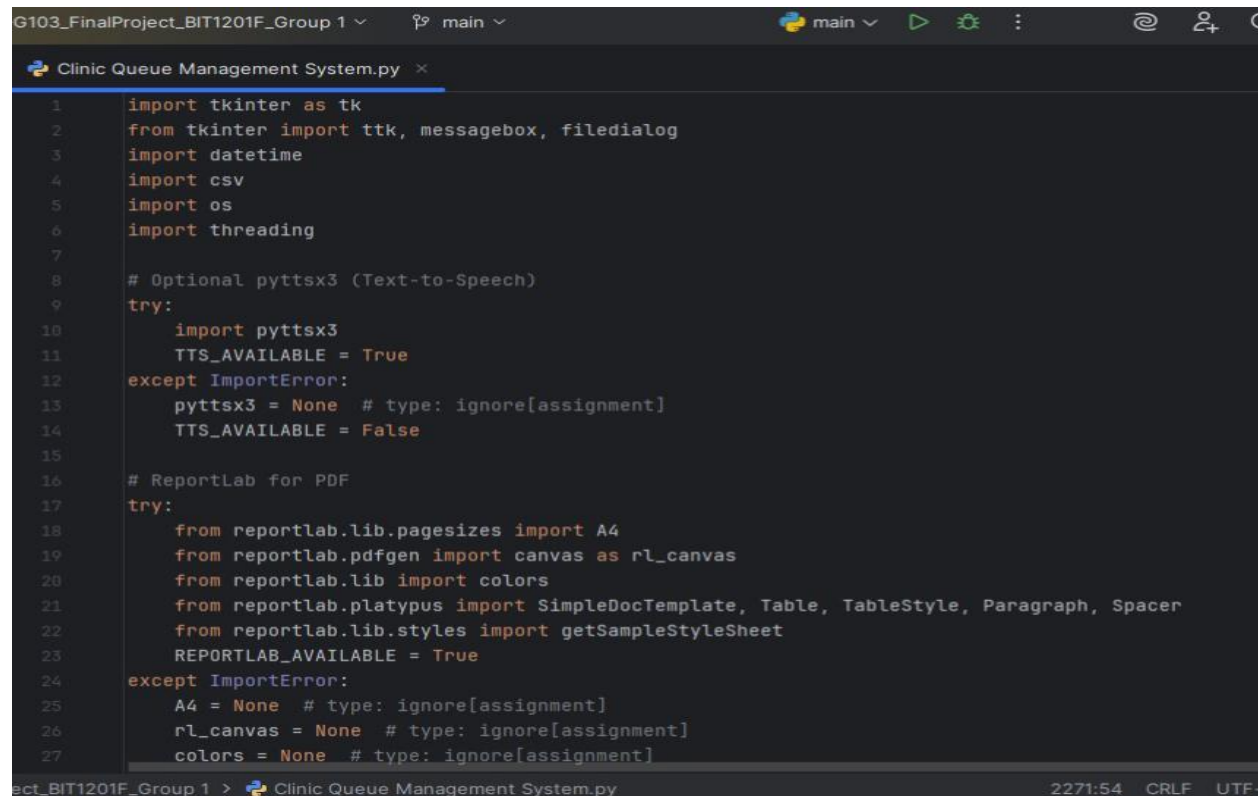
Once logged in, a member of staff is presented with a dashboard summarising the day's activity (total patients, patients waiting, patients served, and available doctors), and a sidebar from which they can navigate to patient registration, queue management, search, the live monitor, doctor management, reports, and — for administrators — the audit log and settings.

At the centre of the system is the registration module. When a patient arrives, a staff member enters their personal details (name, age, gender, phone number, address), selects the department they need to visit (General Consultation, Pediatrics, Maternity, Laboratory, Dental, or Pharmacy), and assigns a priority level (Normal, Elderly, or Emergency). The system automatically generates a department-specific queue number (for example, G001 for the first General Consultation patient), assigns the next available doctor in that department, and records the exact arrival time. If no input is provided for a required field, or if the age entered is not a realistic number, the system blocks registration and displays an inline warning rather than silently failing or crashing.

Once registered, patients enter the live queue, which staff can view, filter by department or status, and update as patients are called, are being served, or have completed their visit. Patients with an Emergency or Elderly priority level are automatically sorted ahead of Normal patients within the same department, ensuring that the most urgent cases are seen first regardless of arrival time. The system also features an optional spoken voice announcement (using the pyttsx3 text-to-speech engine) that calls out a patient's name and queue number, mirroring the announcement systems used in larger hospitals.

Administrative and reporting features round out the system: a Reports module allows staff to generate and export daily statistics as CSV, plain text, or a formatted PDF queue ticket and summary report (using the reportlab library); an Audit Log records every significant action taken in the system (logins, registrations, status changes, user management) with a timestamp, the responsible user, and a description of the action, supporting accountability and traceability; and a Settings module allows an administrator to adjust clinic-wide parameters such as operating hours, maximum daily patient capacity, and interface preferences such as dark mode and font size.

1.3 Structured Programming Application



```
1 import tkinter as tk
2 from tkinter import ttk, messagebox, filedialog
3 import datetime
4 import csv
5 import os
6 import threading
7
8 # Optional pytsx3 (Text-to-Speech)
9 try:
10     import pytsx3
11     TTS_AVAILABLE = True
12 except ImportError:
13     pytsx3 = None # type: ignore[assignment]
14     TTS_AVAILABLE = False
15
16 # ReportLab for PDF
17 try:
18     from reportlab.lib.pagesizes import A4
19     from reportlab.pdfgen import canvas as rl_canvas
20     from reportlab.lib import colors
21     from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph, Spacer
22     from reportlab.lib.styles import getSampleStyleSheet
23     REPORTLAB_AVAILABLE = True
24 except ImportError:
25     A4 = None # type: ignore[assignment]
26     rl_canvas = None # type: ignore[assignment]
27     colors = None # type: ignore[assignment]
```

The system was deliberately built using procedural, structured programming techniques rather than object-oriented design, in direct alignment with the module's emphasis on variables, decision structures, iteration, and modular functions rather than classes and objects.

Data is stored using straightforward built-in Python data structures: dictionaries for single records with named fields (such as the current logged-in user, clinic settings, and individual patient and doctor records), and lists for collections of records (the patient queue, the doctor roster, and the audit log). Global counters, implemented as single-element lists to allow modification inside nested functions, track the next available patient ID and the next queue number for each department.

Decision structures (if, elif, else) are used extensively throughout the system: to validate registration form input, to determine whether a doctor is available for a given department, to decide whether a patient's priority warrants jumping the queue, to control which navigation options a user sees based on their role, and to branch the login process between a successful sign-in and an error message.

Iteration (loops) is used to search through the list of doctors to find an available match for a department, to filter the patient list by date, department, or search term, to populate the on-screen queue and report tables row by row, and to calculate daily statistics such as total patients served per department.

Modularity is achieved through a large number of small, purpose-built functions, each responsible for a single task. Examples include `get_next_queue_number()`, which generates the next sequential queue number for a department; `get_available_doctor()`, which searches for and returns a free doctor in the requested department; `priority_sort_key()`, which provides the sort order used to rank patients by urgency;

`add_audit()`, which appends a timestamped entry to the audit log; and a dedicated `build_` function for every screen in the application (`build_dashboard`, `build_register`, `build_queue`, `build_search`, `build_monitor`, `build_doctors`, `build_reports`, `build_audit`, and `build_settings`), each of which constructs and wires up a single page of the interface. This satisfies the requirement of at least three user-defined functions many times over, while keeping each function's responsibility narrow and easy to test or modify independently.

The GUI itself is also intentionally separated by concern: helper functions such as `make_card()`, `make_btn()`, `make_treeview()`, and `stat_card()` generate reusable interface components, so that visual styling logic is written once and reused across every page, rather than being duplicated throughout the codebase.

1.4 SDG Relevance

The system directly supports Sustainable Development Goal 3: Good Health and Well-being, whose targets include strengthening the capacity of all countries for early warning, risk reduction, and management of health risks, and achieving universal health coverage, including access to quality essential healthcare services. By introducing a structured, prioritised queue, the system gives clinics a practical tool to ensure that patients with urgent or emergency conditions, as well as elderly and otherwise vulnerable patients, are seen promptly rather than waiting behind less urgent walk-in cases purely due to arrival order.

The project also supports Sustainable Development Goal 9: Industry, Innovation and Infrastructure, by promoting the adoption of low-cost, locally adaptable digital infrastructure in a public service setting that has traditionally depended on manual paper processes. Because the application requires no internet connection, no paid database engine, and runs on modest hardware, it is well suited to the resource constraints commonly faced by community clinics in Sierra Leone, supporting the broader goal of building resilient infrastructure and fostering innovation appropriate to the local context.

Finally, by maintaining an audit trail of every action taken in the system and supporting structured, exportable reporting, the project touches on Sustainable Development Goal 16: Peace, Justice and Strong Institutions, in particular the objective of developing effective, accountable, and transparent institutions, since clinic management and oversight bodies gain a verifiable record of daily operations rather than relying on informal or undocumented processes.

2. System Overview

2.1 Application Type

The application is a single-file, GUI-based desktop program written entirely in Python, using the tkinter standard library for the interface and two optional third-party libraries: pyttsx3 for offline text-to-speech voice announcements, and reportlab for generating PDF queue tickets and reports. Both optional libraries are wrapped in try/except import blocks so that the core application remains fully functional even if they are not installed, with the relevant features simply disabled and the user notified.

2.2 User Roles and Access Control

Role	Default Login	Access
Administrator	admin / admin123	Full access: all modules, user management, audit log, and system settings
Receptionist	reception / reception123	Register patients, manage queue, search, view reports
Doctor	doctor / doctor123	View live monitor, search records, view doctor availability

2.3 Core Modules

- Login & Session Management – authenticates users against the USERS dictionary and stores the active session in the current_user dictionary.
- Dashboard – displays live statistics: total patients today, patients waiting, patients served, and available doctors.
- Patient Registration – captures new patient details, validates input, and adds the patient to the queue.
- Queue Management – displays the active queue, allows status updates (Waiting → Called → Serving → Completed), and supports priority-based reordering.
- Search – allows staff to locate a patient record by name, queue number, or phone number.
- Live Monitor – a simplified, auto-refreshing public-facing display of the current queue, suitable for a waiting-room screen.
- Doctor Management – lists all doctors, their department, and current availability status.
- Reports – generates daily statistics and exports them as CSV, plain text, or PDF.
- Audit Log – a chronological record of every significant action performed in the system (administrator only).
- Settings – allows an administrator to configure clinic name, operating hours, daily capacity limits, and interface preferences (administrator only).

3.Functional Requirements Mapping

The table below maps each functional requirement from the project brief to the specific part of the system that satisfies it.

Requirement	Implementation
Entry of relevant user data	Patient Registration form captures name, age, gender, phone, address, department, priority, and notes.
Multiple record handling	All patients are stored as dictionaries inside the global patients list, supporting an unlimited number of concurrent records.
Logical calculations	Queue numbers, daily totals, served counts, and department statistics are calculated from the patient list on demand.
Decision structures (if/elif/else)	Used in form validation, doctor availability lookup, priority sorting, role-based navigation, and login handling.
Loops where necessary	Used to search for available doctors, filter and sort the patient queue, populate tables, and compute report statistics.
Clear, well-formatted, user-friendly output	Colour-coded status badges, formatted statistic cards, structured table views (Treeview), and pop-up notification dialogs.
GUI: labels, entry fields, buttons, output area	Implemented throughout using tkinter Label, Entry, Combobox, Button, and Treeview widgets across every page.
Variables and constants	Session state, theme colours, department prefixes, and clinic settings are defined as module-level variables/constants.
Data types	Strings (names, departments), integers (age, counters), floats (voice volume), booleans (dark mode, voice enabled), dictionaries, and lists are all used.
At least three user-defined functions	The system defines well over thirty functions, including <code>get_next_queue_number()</code> , <code>get_available_doctor()</code> , <code>add_audit()</code> , and a dedicated build function for every screen.

4. Data, Privacy & Compliance

4.1 Data Accessibility

Patient and operational data is held in memory during a session and can be exported on demand into open, widely supported formats: CSV (comma-separated values) for spreadsheet analysis, plain text for quick review, and PDF for printable tickets and reports. This ensures that clinic staff are never locked into a proprietary format and can move data into other systems (such as a national health information system) if required.

4.2 Interoperability

Because exports use plain CSV and PDF rather than a proprietary database format, the data produced by the system can be opened, read, and processed by virtually any spreadsheet application, text editor, or PDF viewer, supporting interoperability with other tools a clinic or district health office may already use.

4.3 Privacy Considerations

Access to patient data is restricted by role: only Administrators and Receptionists can register new patients or view full reports, while Doctors can view queue and search information relevant to patient care without administrative access. Every login is required before any patient data can be viewed or modified, and the system maintains no persistent storage of plaintext passwords beyond the in-memory session for the duration the application is open.

4.4 Ethical Use of User Data

The audit log records every sensitive action — logins, logouts, patient registrations, status changes, and user account management — together with the responsible staff member and a timestamp. This provides accountability and discourages misuse of patient information, supporting the ethical handling of personal health data even within a lightweight, offline desktop tool.

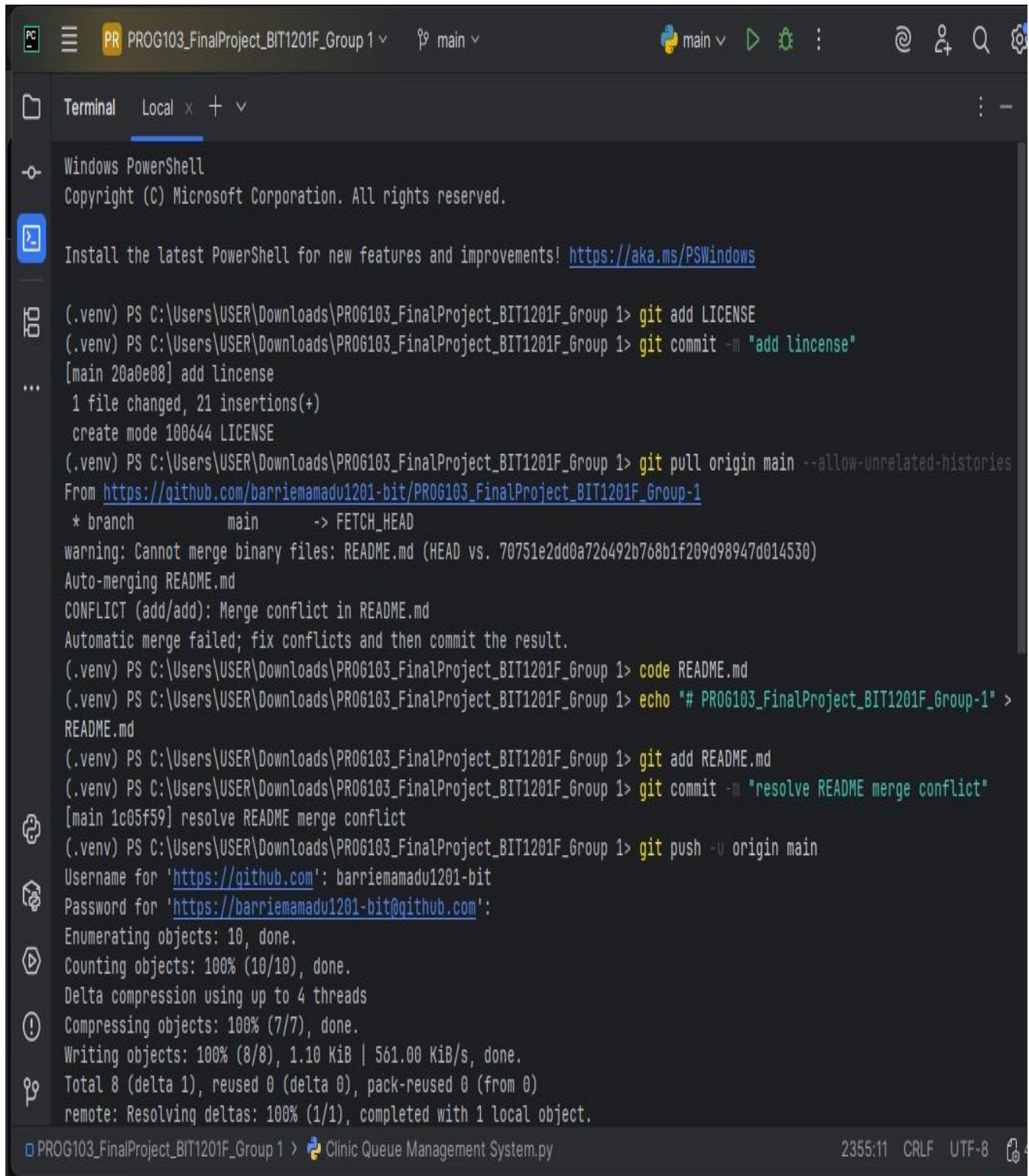
5. Quality and Design Practices

To meet the project's advanced and quality requirements, the following practices were applied throughout development:

- Input validation – the registration form rejects empty required fields and non-numeric or unrealistic age values before a patient can be added to the queue.
- Meaningful naming – functions and variables are named to reflect their purpose (for example, `get_available_doctor`, `priority_sort_key`, `dept_counters`), making the codebase easier to read and maintain.
- Separation of concerns – reusable interface-building helpers (`make_card`, `make_btn`, `make_treeview`) are kept separate from page-specific logic, and business logic (queue numbering, doctor assignment, priority sorting) is kept separate from the widgets that display it.
- Comments and section headers – the source file is divided into clearly labelled sections (Global Data Storage, Login Screen, Page: Dashboard, Page: Patient Registration, and so on), making navigation straightforward for anyone reviewing or extending the code.
- Graceful degradation – optional dependencies (`pyttsx3` for voice, `reportlab` for PDF export) are imported defensively, so the application continues to run with reduced functionality rather than crashing if a library is missing.
- Consistent visual design – a centralised theme dictionary (light and dark variants) ensures consistent colours, spacing, and styling across every page, and supports a one-click dark mode toggle.

6. GitHub and Version Control

The project source code, README.md, and an MIT open-source LICENSE file have been published to a public GitHub repository, in line with the module's open-source and standards requirements. Version control was carried out entirely through the integrated terminal, using the following sequence of commands:



```
PR PROG103_FinalProject_BIT1201F_Group 1 main
Terminal Local x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

(.venv) PS C:\Users\USER\Downloads\PROG103_FinalProject_BIT1201F_Group 1> git add LICENSE
(.venv) PS C:\Users\USER\Downloads\PROG103_FinalProject_BIT1201F_Group 1> git commit -m "add lincense"
[main 20a0e08] add lincense
1 file changed, 21 insertions(+)
create mode 100644 LICENSE
(.venv) PS C:\Users\USER\Downloads\PROG103_FinalProject_BIT1201F_Group 1> git pull origin main --allow-unrelated-histories
From https://github.com/barriemamaduv1201-bit/PROG103\_FinalProject\_BIT1201F\_Group-1
* branch      main      -> FETCH_HEAD
warning: Cannot merge binary files: README.md (HEAD vs. 70751e2dd0a726492b768b1f209d98947d014530)
Auto-merging README.md
CONFLICT (add/add): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
(.venv) PS C:\Users\USER\Downloads\PROG103_FinalProject_BIT1201F_Group 1> code README.md
(.venv) PS C:\Users\USER\Downloads\PROG103_FinalProject_BIT1201F_Group 1> echo "# PROG103_FinalProject_BIT1201F_Group-1" > README.md
(.venv) PS C:\Users\USER\Downloads\PROG103_FinalProject_BIT1201F_Group 1> git add README.md
(.venv) PS C:\Users\USER\Downloads\PROG103_FinalProject_BIT1201F_Group 1> git commit -m "resolve README merge conflict"
[main 1c05f59] resolve README merge conflict
(.venv) PS C:\Users\USER\Downloads\PROG103_FinalProject_BIT1201F_Group 1> git push -u origin main
Username for 'https://github.com': barriemamaduv1201-bit
Password for 'https://barriemamaduv1201-bit@github.com':
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 4 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (8/8), 1.10 KiB | 561.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
```

PROG103_FinalProject_BIT1201F_Group 1 > Clinic Queue Management System.py 2355:11 CRLF UTF-8

7. Conclusion

The Clinic Queue Management System demonstrates the practical application of structured programming principles — variables and constants, multiple data types, decision structures, iteration, and modular functions — within a real, GUI-based desktop application built using Python's tkinter library. Beyond satisfying the technical requirements of the PROG103 module, the system addresses a genuine operational problem faced by health facilities in Sierra Leone: the inefficiency and lack of prioritisation inherent in manual, paper-based patient queues.

By introducing role-based access, automatic priority sorting, exportable reporting, and a full audit trail, the system offers a low-cost, locally deployable digital alternative that directly supports Sustainable Development Goal 3 (Good Health and Well-being), while also touching on Goals 9 and 16 through its support for accessible infrastructure and accountable record-keeping. The project has been documented and published as an open-source repository on GitHub under the MIT License, fulfilling the module's open-source, documentation, and version-control requirements.

8. References

- GitHub (2026) *GitHub: Build and ship software on a single platform*. Available at: <https://github.com> (Accessed: 18 June 2026).
- Python Software Foundation (2026a) *Python Documentation*. Available at: <https://docs.python.org/3/> (Accessed: 18 June 2026).
- Python Software Foundation (2026b) *Tkinter — Python interface to Tcl/Tk*. Available at: <https://docs.python.org/3/library/tkinter.html> (Accessed: 18 June 2026).
- United Nations (2026a) *Goal 3: Good health and well-being*. Available at: <https://sdgs.un.org/goals/goal3> (Accessed: 18 June 2026).
- Summerfield, M. (2018) *Rapid GUI Programming with Python and Qt*. Upper Saddle River, NJ: Prentice Hall. [Suggested for GUI design concepts]